Clasificator Multinomial Naive Bayes pentru Detecția Anunțurilor de Angajare False

Ștefan Chiper

16 noiembrie 2025

Cuprins

1 Introducere **2**

2 Modelul Matematic Folosit **2**

2.1 Teorema lui Bayes 2

2.2 Ipoteza Naivă 2

2.3 Formula de Decizie 3

2.4 Calculul Probabilităților (Antrenarea) 3

2.4.1 Probabilitățile Apriori 3

2.4.2 Probabilitățile Condiționate și Netezirea Laplace 3

3 Structura Codului și Funcțiile Principale **3**

3.1 Clasa `MultinomialNaiveBayes` 3

3.2 Funcțiile de Pre-procesare (Feature Engineering) 4

3.3 Scriptul Principal de Execuție 4

4 Instrucțiuni de Utilizare **4**

4.1 Dependințe 4

4.2 Rularea Scriptului 4

5 Exemplu de Utilizare (Output) **5**

6 Referințe Bibliografice **6**

1. Introducere

Acest document detaliază implementarea unui clasificator de text de tip **Multinomial Naive Bayes (MNB)** de la zero, folosind limbajul Python. Scopul principal al proiectului este de a antrena un model capabil să facă distincția între anunțuri de angajare legitime (reale) și cele frauduloase (false), pe baza unui set de date CSV.

Proiectul adoptă o abordare hibridă:

- **Nucleul Modelului:** Clasa `MultinomialNaiveBayes` este implementată de la zero, folosind doar `numpy` și `pandas` pentru procesarea datelor.
- **Evaluarea și Biblioteci:** Se utilizează biblioteca `scikit-learn` pentru împărțirea setului de date (`train_test_split`) și generarea metricilor textuale (`classification_report`), alături de `matplotlib` și `seaborn` pentru generarea unei matrici de confuzie vizuale.

Setul de date utilizat, `fake_job_postings.csv`, conține o varietate de câmpuri textuale și cu date categoriale, care sunt procesate și combinate pentru a forma vocabularul modelului.

2. Modelul Matematic Folosit

Algoritmul Naive Bayes este o metodă de clasificare probabilistică bazată pe **Teorema lui Bayes**, cu o presupunere naivă de independență condiționată între caracteristici (cuvinte).

2.1. Teorema lui Bayes

Formula de bază pentru clasificarea unui document d într-o clasă c (ex: 'Real' sau 'Fake') este:

$$P(c|d) = \frac{P(d|c) \cdot P(c)}{P(d)}$$

Unde:

- $P(c|d)$ este probabilitatea **aposteriori** (probabilitatea ca documentul d să aparțină clasei c).
- $P(d|c)$ este probabilitatea **condiționată** (probabilitatea de a observa documentul d știind că aparține clasei c).
- $P(c)$ este probabilitatea **apriori** (probabilitatea generală a clasei c în setul de date).
- $P(d)$ este probabilitatea documentului d (acționează ca o constantă de normalizare și poate fi ignorată în clasificare).

2.2. Ipoteza Naivă

Deoarece un document d este format dintr-o colecție de cuvinte (token-uri) $w_1, w_2, ..., w_n$, calculul lui $P(d|c)$ devine $P(w_1, ..., w_n|c)$. Ipoteza naivă presupune că apariția fiecărui cuvânt este independentă de celelalte, dată fiind clasa:

$$P(d|c) \approx \prod_{i=1}^{n} P(w_i|c)$$

2.3. Formula de Decizie

Clasificarea finală se reduce la a găsi clasa c care maximizează probabilitatea aposteriori (funcția `argmax`):

$$c_{best} = \arg\max_c P(c) \cdot \prod_{i=1}^{n} P(w_i|c)$$

Pentru a evita problemele de *underflow* numeric (înmulțirea multor probabilități foarte mici), se trece în domeniul logaritmic, transformând produsele în sume:

$$c_{best} = \arg\max_c \left(\log(P(c)) + \sum_{i=1}^{n} \log(P(w_i|c)) \right)$$

2.4. Calculul Probabilităților (Antrenarea)

Modelul "învață" prin calcularea valorilor $\log(P(c))$ și $\log(P(w_i|c))$ din setul de antrenament.

2.4.1. Probabilitățile Apriori

$$P(c) = \frac{N_c}{N_{total}}$$

2.4.2. Probabilitățile Condiționate și Netezirea Laplace

$$P(w_i|c) = \frac{\text{count}(w_i, c) + \alpha}{\text{count}(c) + (\alpha \cdot |V|)}$$

Unde $\alpha = 1$ este netezirea Laplace.

3. Structura Codului și Funcțiile Principale

Scriptul este împărțit în trei părți: clasa modelului, funcțiile de pre-procesare și scriptul principal de execuție.

3.1. Clasa `MultinomialNaiveBayes`

- **`__init__(self, alpha=1.0)`**: Inițializează hiperparametrul de netezire `alpha`.
- **`_preprocess_text(self, text)`**: O funcție de tokenizare simplificată (`text.split()`).
- **`fit(self, X, y)`**: Funcția de antrenare, care calculează $\log(P(c))$ și $\log(P(w_i|c))$.
- **`predict(self, X)`**: Funcția de clasificare, care aplică formula de decizie log-probabilistică.

3.2. Funcțiile de Pre-procesare (Feature Engineering)

- **`clean_natural_text(text)`**: Aplicată coloanelor textuale. Transformă în litere mici, înlocuiește numerele cu `__NUMAR__`, elimină punctuația și cuvintele de oprire (stop words).
- **`tokenize_categorical(column_name, value)`**: Aplicată coloanelor cu date categoriale. Transformă o valoare (ex: 'IT') într-un token unic (ex: `TOKEN_INDUSTRY_IT`) și gestionează valorile nule (ex: `TOKEN_INDUSTRY_NA`).

3.3. Scriptul Principal de Execuție

1. **Încărcarea Datelor:** Citește `fake_job_postings.csv`.
2. **Procesarea Tuturor Coloanelor:** Combină coloanele `TEXT_COLS` și `CAT_COLS` (după curățare și tokenizare) într-o singură coloană `full_text`.
3. **Împărțirea Setului de Date:** Folosește `train_test_split` cu `stratify=y` și `random_state=42` pentru reproductibilitate.
4. **Antrenarea și Evaluarea:** Inițializează și antrenează clasa `MultinomialNaiveBayes`. Generează predicții și afișează raportul `classification_report` și heatmap-ul `confusion_matrix`.

4. Instrucțiuni de Utilizare

4.1. Dependințe

Pentru a rula scriptul, este necesar un mediu Python 3 și următoarele biblioteci:

- `pandas`
- `numpy`
- `scikit-learn`
- `matplotlib`
- `seaborn`

Acestea pot fi instalate folosind `pip`:

4.2. Rularea Scriptului

1. Salvați codul Python într-un fișier (de ex: `train_classifier.py`).
2. Plasați setul de date `fake_job_postings.csv` în exact același director cu fișierul `.py`.
3. Deschideți un terminal sau o linie de comandă în acel director.
4. Executați scriptul.

5. Exemplu de Utilizare (Output)

La rularea scriptului, acesta va afișa progresul în consolă și va genera un fișier imagine. Output-ul text va fi similar cu următorul:

```
Se incarca datele...
Se pre-proceseaza si se combina toate coloanele...
Se curata coloanele text...
Se tokenizeaza coloanele cu date categoriale...
Am incarcat si procesat 17880 randuri.
Se impart datele folosind train_test_split...
Date impartite: 13410 esantioane de antrenare, 4470 esantioane de testare.
Incepe antrenarea (fit)...
Antrenare: Vocabular descoperit cu 143041 cuvinte unice.
Antrenare: Se calculeaza probabilitatile Apriori P(c)...
Antrenare: Se calculeaza probabilitatile Conditionate P(w|c)...
Antrenarea s-a incheiat cu succes!

--- Evaluarea Modelului pe Setul de Test (cu sklearn.metrics) ---
Acuratete (Accuracy): 97.32%

Se genereaza Matricea de Confuzie Avansata (matrice_confuzie.png)...
Imaginea a fost salvata ca 'matrice_confuzie.png'
```

Scriptul generează fișierul `matrice_confuzie.png` (Figura 1), care conține o reprezentare vizuală a performanței modelului.

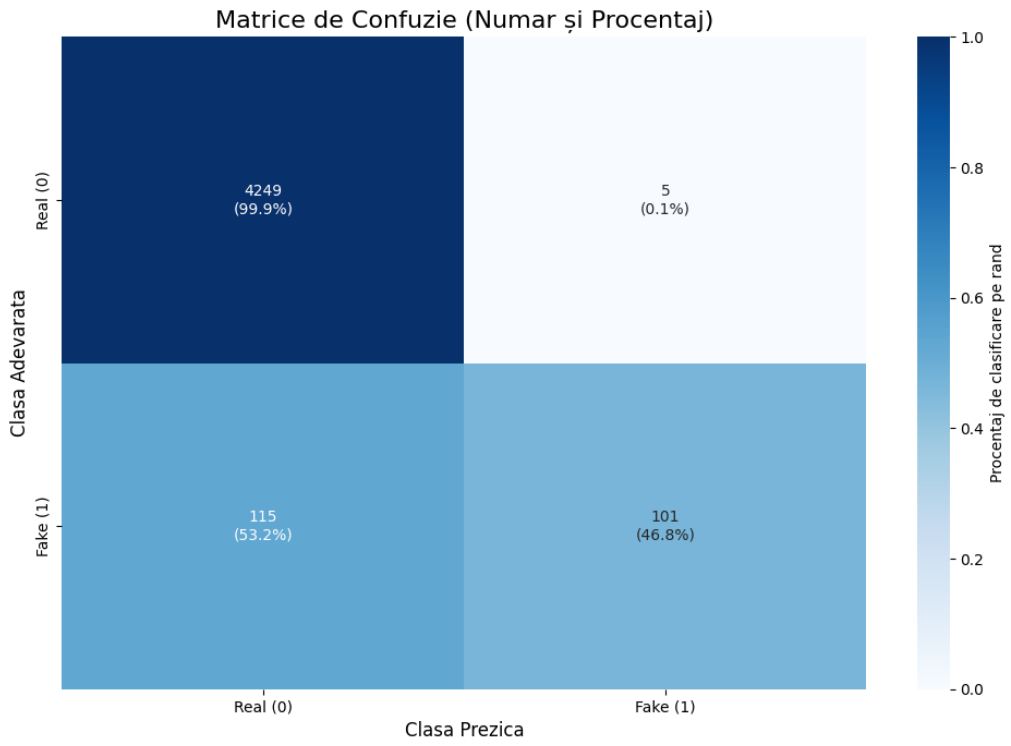

Figura 1: Exemplu de matrice de confuzie generată de script, arătând atât numărul absolut, cât și procentajul normalizat pe rând.

6. Referințe Bibliografice

Bibliografie

[1] Shivam Bansal, (2020), *Real / Fake Job Posting Prediction* [Set de date Kaggle]
https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction

[2] Scikit-learn Developers, (2025), *Naive Bayes - Scikit-learn Documentation.*
Disponibil online: https://scikit-learn.org/stable/modules/naive_bayes.html

[3] StatQuest with Josh Starmer, (2020), *Naive Bayes, Clearly Explained!!!*
Disponibil online: https://www.youtube.com/watch?v=O2L2Uv9pdDA

[4] StatQuest with Josh Starmer, (2020), *Gaussian Naive Bayes, Clearly Explained!!!*
Disponibil online: https://www.youtube.com/watch?v=H3EjCKtlVog